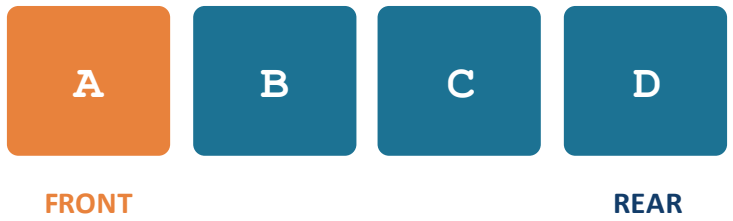
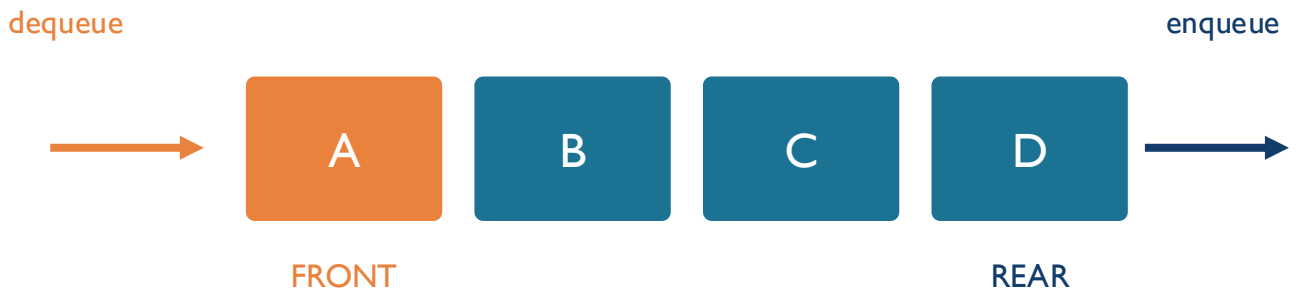
Queues



What Is a Queue?

- A linear structure where elements are added at the rear and removed from the front
- Follows the FIFO rule: First In, First Out
- The first element inserted is always the first one to leave



A Familiar Analogy

- A queue behaves exactly like a real waiting line
- New arrivals join at the back of the line
- Only the person at the front is served next
- No one can cut in — order of arrival is preserved

Core Queue Operations

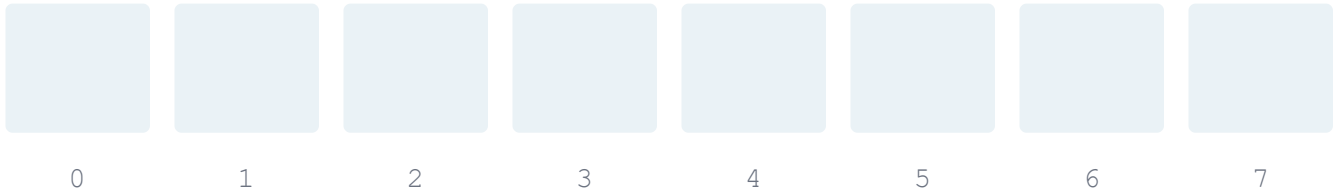
enqueue(x)	Insert element x at the rear
dequeue()	Remove and return the front element
peek()	View the front element without removing it
isEmpty()	Check whether the queue has zero elements

Array Queue — Struct & initQueue()

- The simplest queue is just an array plus two markers
- front is the index of the next element to remove
- rear is the index of the last element inserted

```
#define MAX 8
typedef struct {
    int items[MAX];
    int front, rear;
} Queue;

void initQueue(Queue *q) {
    q->front = 0;
    q->rear = -1;
}
```



An empty array of capacity 8. Exit and entry points are initialized as: front = 0, rear = -1

Array Queue — enqueue()

- Checks for overflow: the the array is full once rear rear reaches the last index
- Otherwise, rear moves one step forward
- The new value is written at the updated rear position

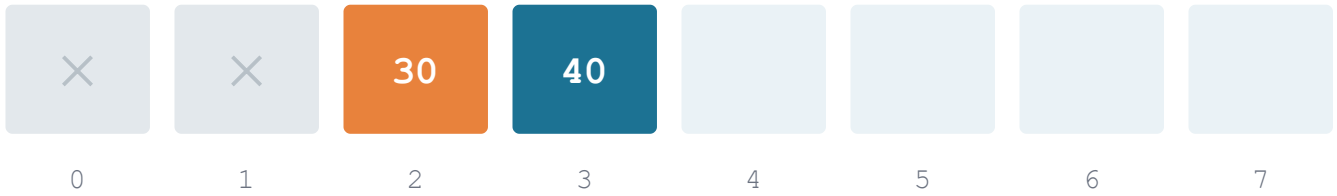
```
void enqueue(Queue *q, int value) {
    if (q->rear == MAX - 1) {
        printf("Queue Overflow\n");
        return;
    }
    q->items[++(q->rear)] = value;
}
```

Array Queue — dequeue() Limitation

- Checks for underflow: empty once front passes passes rear
- Otherwise, it reads items[front] and moves front forward

```
int dequeue(Queue *q) {
    if (q->front > q->rear) {
        printf("Queue Underflow\n");
        return -1;
    }
    return q->items[(q->front)++];
}
```

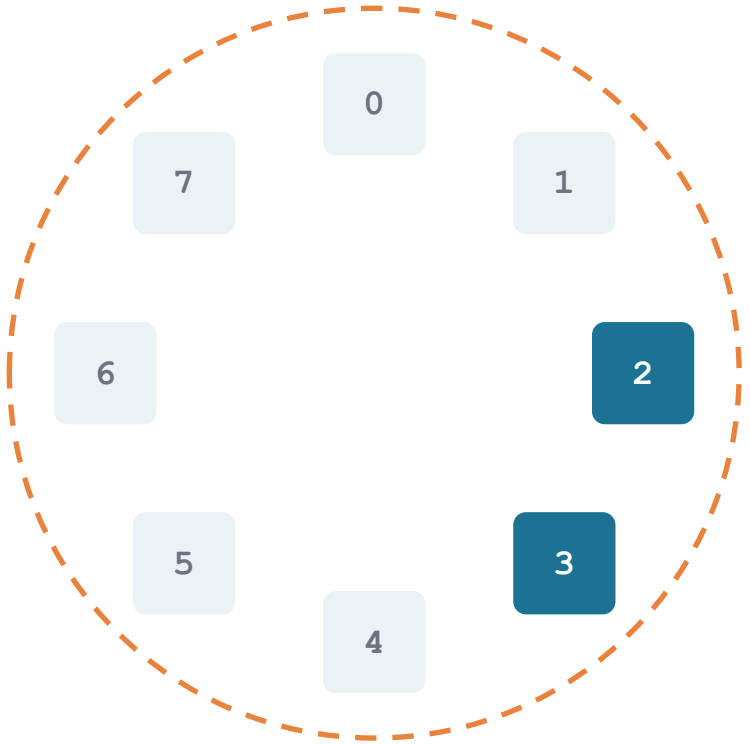
front = 2 (after 2 calls to dequeue)



⚠ Indices 0 and 1 are now unused, but this simple version can never reuse reuse them.

The Fix: a Circular Queue

- A circular queue wraps the rear back to index 0 once it reaches the the end
- Freed slots at the start of the array get reused instead of wasted
- This wrap-around is done with modulo arithmetic:
- rear = (rear + 1) % capacity



The next enqueue wraps to index 0

Circular Queue — Struct & initQueue()

- front, rear, and count together track the queue’s state
- count avoids ambiguity between a full and an empty queue
- initQueue() resets the queue before first use

```
#define MAX 8
typedef struct {
    int items[MAX];
    int front, rear, count;
} Queue;

void initQueue(Queue *q) {
    q->front = 0; q->rear = -1;
    q->count = 0;
}
```

Circular Queue — enqueue()

- Checks for overflow using using count instead of rear rear
- rear advances with % MAX, wrapping to 0 at capacity
- count increases by one after every successful insert

```
void enqueue(Queue *q, int value) {
    if (q->count == MAX) {
        printf("Queue Overflow\n");
        return;
    }
    q->rear = (q->rear + 1) % MAX;
    q->items[q->rear] = value;
    q->count++;
}
```

Circular Queue — dequeue()

- Checks for underflow using count
- front advances with % MAX, wrapping to 0 at capacity
- count decreases by one after every successful removal

```
int dequeue(Queue *q) {
    if (q->count == 0) {
        printf("Queue Underflow\n");
        return -1;
    }
    int value = q->items[q->front];
    q->front = (q->front + 1) % MAX;
    q->count--;
    return value;
}
```

Circular Queue — peek() & isEmpty()

- peek() reads the front value without removing it it
- isEmpty() is a one-line helper based on count
- Both run in constant time, O(1)

```
int peek(Queue *q) {
    if (q->count == 0) return -1;
    return q->items[q->front];
}
int isEmpty(Queue *q) {
    return q->count == 0;
}
```

Uses of Queues

- Print spooling (Think of a shared printer)
- CPU task scheduling (One CPU, many jobs)
- Customer supports (limited employees, many issues)
- Data buffering (Streaming systems)
- Message queues (modern systems are interconnects of smaller components)
- Ride and delivery matching (queue nearby requests before matching them to a driver)

Time Complexity

- Time complexity is O(1) for enqueue, dequeue, peek.
  - Under the assumption that we have front and rear indices to directly access the positions where the addition/deletions will happen
  - Also, no shifting of other elements is ever required.

- What happens for linked list based implementation for queue?

Linked-list based implementation

```c
#include <stdio.h>
#include <stdlib.h>

// Node structure for the linked list
struct Node
{
    int data;
    struct Node* next;
};

// Queue structure with front and rear pointers
struct Queue
{
    struct Node* front;
    struct Node* rear;
};

// Initialize an empty queue
void initQueue(struct Queue* q)
{
    q->front = NULL;
    q->rear = NULL;
}
```

```c
// Enqueue: insert element at the rear
void enqueue(struct Queue* q, int value)
{
        struct Node* newNode = (struct Node*) malloc
                                (sizeof(struct Node));
        if (newNode == NULL) {
            printf("Memory allocation failed!\n");
            return;
        }
        newNode->data = value;
        newNode->next = NULL;
        if (isEmpty(q)) {
        // If queue was empty before this enqueue, then
        // new node should be front and rear
            q->front = newNode;
            q->rear = newNode;
        } else {
        // Link new node at the end, update rear
            q->rear->next = newNode;
            q->rear = newNode;
        }
        printf("Enqueued: %d\n", value);
}
```

Linked-list based implementation

```c
// Dequeue: remove element from the front
int dequeue(struct Queue* q)
{
        if (isEmpty(q))
        {
            printf(”Empty Queue! Can’t dequeue.\n”);
            return -1; // Special value for failure
        }

        struct Node* temp = q->front;
        int dequeuedValue = temp->data;
        q->front = q->front->next;

        // If queue becomes empty after dequeue,
        // update rear too
        if (q->front == NULL) {
            q->rear = NULL;
        }
        free(temp);
        return dequeuedValue;
}
```

```c
// Print queue elements from front to rear
void printQueue(struct Queue* q)
{
        if (isEmpty(q))
        {
            printf("Queue is empty.\n");
            return;
        }

        struct Node* current = q->front;
        printf("Queue (front to rear): ");

        while (current != NULL)
        {
            printf("%d ", current->data);
            current = current->next;
        }
        printf("\n");
}
```